

Restaurant

Database Management System

Database Schema Documentation

Database Name: SingleRestaurantDB

RDBMS: MySQL

1. Overview

SingleRestaurantDB is a relational database designed to manage the core operations of a single-branch restaurant. It tracks customers, the food menu, orders placed by customers, the line items within each order, and payments made against orders. The schema uses standard relational integrity: primary keys autoincrement on each table, and foreign keys link child records back to their parent entities.

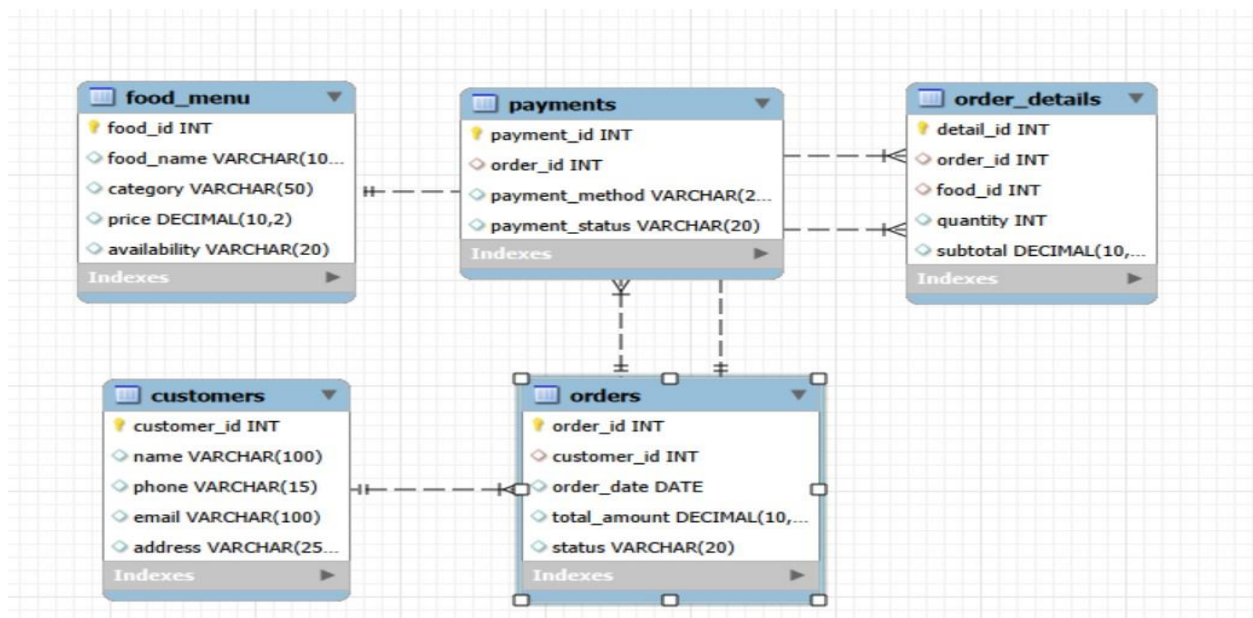
The database consists of five tables:

- Customers — stores customer contact and address details
- Food_Menu — stores menu items, categories, prices, and availability
- Orders — stores order-level data linked to a customer
- Order_Details — stores individual line items (food + quantity) within an order
- Payments — stores payment method and status for an order

2. Entity Relationship Overview

The relationships between tables are as follows:

- Customers (1) → (Many) Orders — one customer can place many orders
 - Orders (1) → (Many) Order_Details — one order can contain many food items
 - Food_Menu (1) → (Many) Order_Details — one menu item can appear in many order details
 - Orders (1) → (1) Payments — one order has one associated payment record
- Relationship diagram:



3. Table Definitions

3.1 Customers

Stores registered customer information used to associate orders with a person.

Column	Data Type	Constraints / Notes
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	Customer full name
phone	VARCHAR(15)	Contact phone number
email	VARCHAR(100)	Customer email address
address	VARCHAR(255)	Delivery / contact address

Notes:

- Referenced by Orders.customer_id (one-to-many).

3.2 Food_Menu

Stores all food items offered by the restaurant, including price and availability.

Column	Data Type	Constraints / Notes
food_id	INT	PRIMARY KEY, AUTO_INCREMENT
food_name	VARCHAR(100)	Name of the food item
category	VARCHAR(50)	e.g. Burger, Pizza, Sides
price	DECIMAL(10,2)	Unit price of the item
availability	VARCHAR(20)	e.g. Available / Unavailable

Notes:

- Referenced by Order_Details.food_id (one-to-many).

3.3 Orders

Stores order-level information for each transaction placed by a customer.

Column	Data Type	Constraints / Notes
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_id	INT	FOREIGN KEY → Customers(customer_id)
order_date	DATE	Date the order was placed
total_amount	DECIMAL(10,2)	Total value of the order
status	VARCHAR(20)	e.g. Delivered, Pending, Cancelled

Notes:

- Each order belongs to exactly one customer.
- Referenced by Order_Details.order_id and Payments.order_id.

3.4 Order_Details

Stores the individual food items and quantities that make up each order (the order's line items).

Column	Data Type	Constraints / Notes
detail_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FOREIGN KEY → Orders(order_id)
food_id	INT	FOREIGN KEY → Food_Menu(food_id)
quantity	INT	Number of units of the food item ordered
subtotal	DECIMAL(10,2)	quantity × unit price for this line item

Notes:

- Acts as a junction table resolving the many-to-many relationship between Orders and Food_Menu.

3.5 Payments

Stores payment details associated with an order.

Column	Data Type	Constraints / Notes
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FOREIGN KEY → Orders(order_id)
payment_method	VARCHAR(20)	e.g. Cash, Card, Online
payment_status	VARCHAR(20)	e.g. Paid, Pending, Refunded

Notes:

- Each order is expected to have one corresponding payment record.

4. Foreign Key Relationships Summary

Child Table	Child Column	Parent Table	Parent Column
Orders	customer_id	Customers	customer_id
Order_Details	order_id	Orders	order_id
Order_Details	food_id	Food_Menu	food_id
Payments	order_id	Orders	order_id

5. Sample Data

The following sample records are inserted into the database to demonstrate its use.

5.1 Customers

customer_id	name	phone	email	address
1	Ali	03001234567	ali@gmail.com	Abbottabad
2	Sara	03123456789	sara@gmail.com	Islamabad

5.2 Food_Menu

food_id	food_name	category	price	availability
1	Zinger Burger	Burger	500.00	Available
2	Chicken Pizza	Pizza	1200.00	Available
3	Fries	Sides	200.00	Available

5.3 Orders

order_id	customer_id	order_date	total_amount	status
1	1	2026-05-20	1700.00	Delivered

5.4 Order_Details

detail_id	order_id	food_id	quantity	subtotal
1	1	1	1	500.00
2	1	2	1	1200.00

5.5 Payments

payment_id	order_id	payment_method	payment_status	
1	1	Cash	Paid	

6. Key Queries

The following queries are used to retrieve reporting information from the database.

6.1 Customer Orders Summary

Lists each order together with the customer who placed it.

```
SELECT c.name, o.order_id, o.total_amount, o.status FROM Customers c JOIN Orders o ON  
c.customer_id = o.customer_id;
```

6.2 Order Line-Item Detail

Lists each food item within each order, including quantity and subtotal.

```
SELECT o.order_id, f.food_name, od.quantity, od.subtotal FROM Orders o JOIN Order_Details od ON  
o.order_id = od.order_id JOIN Food_Menu f ON od.food_id = f.food_id;
```

6.3 Total Sales

Calculates total revenue across all orders.

```
SELECT SUM(total_amount) AS total_sales FROM Orders;
```

7. Design Notes & Recommendations

- All primary keys use INT AUTO_INCREMENT for simplicity and guaranteed uniqueness.
- Order_Details resolves the many-to-many relationship between Orders and Food_Menu, allowing each order to contain multiple food items.
- total_amount in Orders is currently entered manually on insert; in production it should be derived as the SUM of related Order_Details.subtotal values, ideally via a trigger or application logic, to avoid data drift.
- payment_status and status fields are free-text VARCHAR; consider constraining these with an ENUM or a lookup table (e.g. Order_Status, Payment_Status) to enforce valid values.
- Consider adding UNIQUE constraints on Customers.email and Customers.phone to prevent duplicate customer records.
- Consider adding a created_at / updated_at timestamp column to each table for auditing purposes.